

A Practical Guide to Neural style transfer

TOPIC -- NEURAL STYLE TRANSFER

-- 01 --

$$L_{\text{content}}(p \rightarrow, x \rightarrow, l) = \frac{1}{2} \sum_{i,j} (A_{ij}^l(x \rightarrow) - A_{ij}^l(p \rightarrow))^2$$
$$\frac{1}{2} \sum_{i,j} \left(A_{ij}^l(\vec{x}) - A_{ij}^l(\vec{p}) \right)^2$$

-- 02 --

Symbols Let $x \rightarrow \{\text{vec } x\}$ be an image input to a CNN. Let $F^l \in \mathbb{R}^{N_l \times M_l}$ be the matrix of filter responses in layer l to the image $x \rightarrow \{\text{vec } x\}$, where:

-- 03 --

Style loss The style loss is based on the Gram matrices of the generated and style images, which capture the correlations between different filter responses at different layers of the CNN:

-- 04 --

History NST is an example of image stylization, a problem studied for over two decades within the field of non-photorealistic rendering. The first two example-based style transfer algorithms were image analogies and image quilting. Both of these methods were based on patch-based texture synthesis algorithms. Given a training pair of images—a photo and an artwork depicting that photo—a transformation could be learned and then applied to create new artwork from a new photo, by analogy. If no training photo was available, it would need to be produced by processing the input artwork; image quilting did not require this processing step, though it was demonstrated on only one style. NST was first published in the paper "A Neural Algorithm of Artistic Style" by Leon Gatys et al., originally released to ArXiv 2015, and subsequently accepted by the peer-reviewed CVPR conference in 2016. The original paper used a VGG-19 architecture that has been pre-trained to perform object recognition using the ImageNet dataset. In 2017, Google AI introduced a method that allows a single deep convolutional style transfer network to learn multiple styles at the same time. This algorithm permits style interpolation in real-time, even when done on video media.

-- 05 --

$F_{ijl}(\mathbf{x} \rightarrow)$ is the activation of the i th filter at position j in layer l . A given input image $\mathbf{x} \rightarrow$ is encoded in each layer of the CNN by the filter responses to that image, with higher layers encoding more global features, but losing details on local features.

-- 06 --

Neural style transfer (NST) software algorithms are able to manipulate digital images, or videos, in order to adopt the appearance or visual style of another image. NST algorithms are characterized by their use of deep neural networks for the sake of image transformation. Common uses for NST are the creation of artificial artwork from photographs, for example by transferring the appearance of famous paintings to user-supplied photographs. Several notable mobile apps use NST techniques for this purpose, including DeepArt and Prisma. This method has been used by artists and designers around the globe to develop new artwork based on existent style(s).

-- 07 --

where $A_{ijl}(\mathbf{x} \rightarrow)$ and $A_{ijl}(\mathbf{p} \rightarrow)$ are the activations of the i th filter at position j in layer l for the generated and content images, respectively. Minimizing this loss encourages the generated image to have similar content to the content image, as captured by the feature activations in the chosen layer. The total content loss is a linear sum of the content losses of each layer:
$$L_{\text{content}}(\mathbf{p} \rightarrow, \mathbf{x} \rightarrow) = \sum_l v_l L_{\text{content}}(\mathbf{p} \rightarrow, \mathbf{x} \rightarrow, l)$$
 where the v_l are positive real numbers chosen as hyperparameters.

-- 08 --

Minimizing this loss encourages the generated image to have similar style characteristics to the style image, as captured by the correlations between feature responses in each layer. The idea is that activation pattern correlations between filters in a single layer captures the "style" on the order of the receptive fields at that layer. Similarly to the previous case, the w_l are positive real numbers chosen as hyperparameters.

-- 09 --

Extensions In some practical implementations, it is noted that the resulting image has too much high-frequency artifact, which can be suppressed by adding the

total variation to the total loss. Compared to VGGNet, AlexNet does not work well for neural style transfer. NST has also been extended to videos. Subsequent work improved the speed of NST for images by using special-purpose normalizations. In a paper by Fei-Fei Li et al. adopted a different regularized loss metric and accelerated method for training to produce results in real-time (three orders of magnitude faster than Gatys). Their idea was to use not the pixel-based loss defined above but rather a 'perceptual loss' measuring the differences between higher-level layers within the CNN. They used a symmetric convolution-deconvolution CNN. Training uses a similar loss function to the basic NST method but also regularizes the output for smoothness using a total variation (TV) loss. Once trained, the network may be used to transform an image into the style used during training, using a single feed-forward pass of the network. However the network is restricted to the single style in which it has been trained. In a work by Chen Dongdong et al. they explored the fusion of optical flow information into feedforward networks in order to improve the temporal coherence of the output. Most recently, feature transform based NST methods have been explored for fast stylization that are not coupled to single specific style and enable user-controllable blending of styles, for example the whitening and coloring transform (WCT).

-- 10 --

Training Image $x \rightarrow \{\displaystyle \vec{x}\}$ is initially approximated by adding a small amount of white noise to input image $p \rightarrow \{\displaystyle \vec{p}\}$ and feeding it through the CNN. Then we successively backpropagate this loss through the network with the CNN weights fixed in order to update the pixels of $x \rightarrow \{\displaystyle \vec{x}\}$. After several thousand epochs of training, an $x \rightarrow \{\displaystyle \vec{x}\}$ (hopefully) emerges that matches the style of a $a \rightarrow \{\displaystyle \vec{a}\}$ and the content of $p \rightarrow \{\displaystyle \vec{p}\}$. As of 2017, when implemented on a GPU, it takes a few minutes to converge.